

Converting AI-ML models to Spiking Neural Network Models

Ananya Anilkumar Pawar, Hari Nisanth S | Prof. Joycee Mekie

Department of Electrical Engineering | IIT Gandhinagar

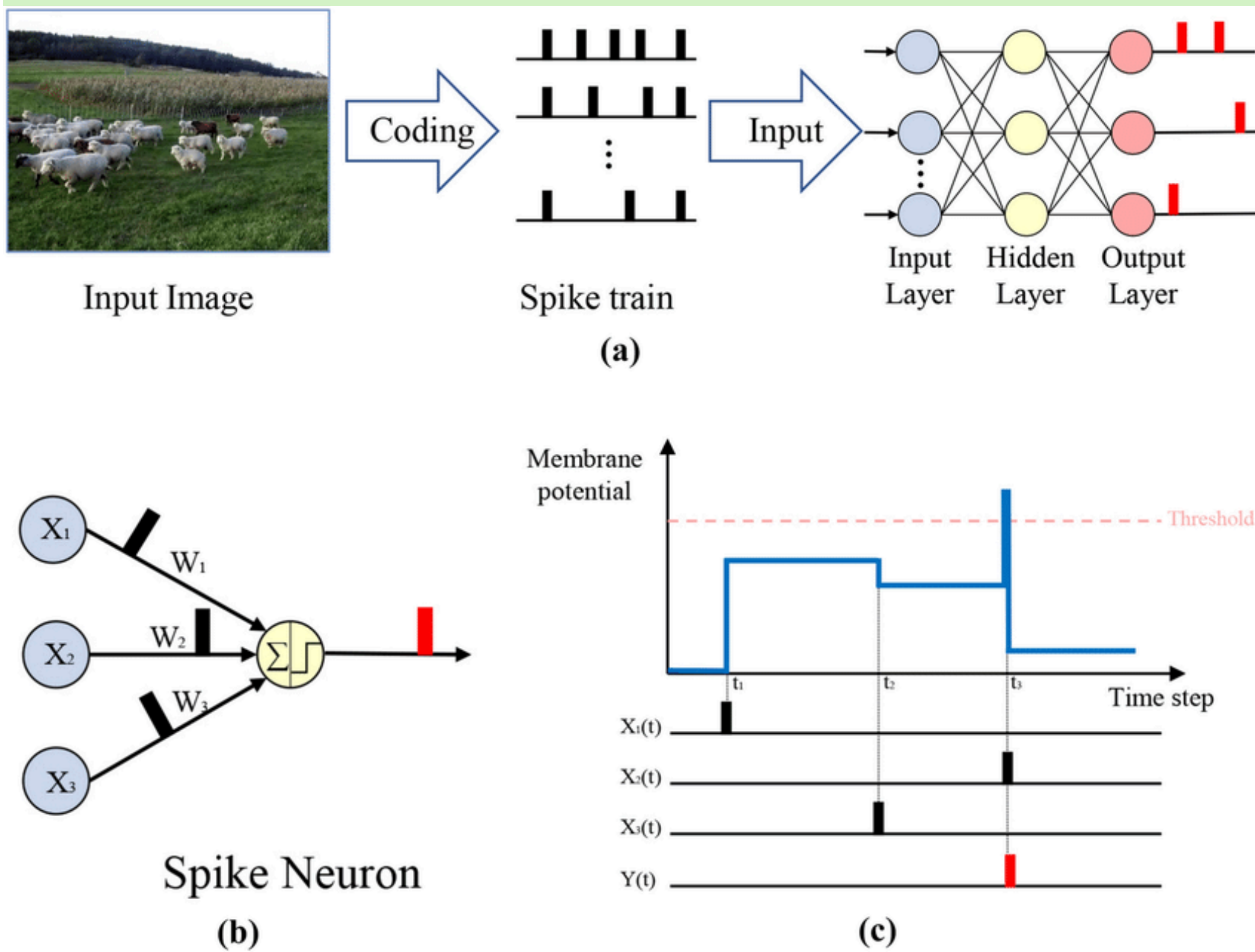
Email: ananya.pawar@iitgn.ac.in, hari.s@iitgn.ac.in



Introduction

- Artificial Neural Networks dominate modern AI due to their high accuracy in tasks like vision and classification. However, they rely on continuous, dense computation, making them energy-intensive and inefficient.
- Spiking Neural Networks (SNNs) are inspired by biological neurons and process information using discrete spikes over time, enabling event-driven and energy-efficient computation.
- This project focuses on the conversion of ANN models into SNNs allowing the reuse of trained models while gaining efficiency benefits.

The motivation for investigating the ANN-SNN conversion is that SNNs consume significantly less energy than ANNs by computing only when neurons spike - making ANN-to-SNN conversion an attractive option to reduce computational cost while maintaining comparable accuracy, with latency being the key tradeoff.



Leaky Integrate and Fire Model

Models like Hodgkin-Huxley closely mimic real biological neurons but their complexity make implementation difficult. The Leaky Integrate-and-Fire (LIF) is thus widely used due to its simplicity and efficiency

A LIF neuron operates in four stages:

- Integrate - accumulates weighted input signals over time, building up membrane potential
- Leak - potential gradually decays in the absence of strong input, mimicking the natural charge loss in biological neurons
- Fire - when the membrane potential $V(t)$ crosses a threshold V_{th} , the neuron emits a spike
- Reset - after firing, the potential returns to a baseline value V_{reset} , ready for the next cycle

$$\tau \frac{dV}{dt} = -(V - V_R) + R_M I,$$

- The SNN uses significantly fewer parameters than CNN while maintaining 98.5% accuracy with reduced model size means lower energy and power consumption.
- Event-Driven Computation: SNNs only compute when spikes occur (sparse, discrete events) versus CNNs that perform dense matrix multiplications every forward pass - this inherent sparsity dramatically reduces unnecessary computations.

How does conversion of an ANN to SNN take place?

- A standard ANN is first trained using ReLU activation and backpropagation to learn optimal weights.
- After training, the learned weights are frozen and transferred to a Spiking Neural Network (SNN).
- The ReLU activation is replaced with spiking neuron models such as Integrate-and-Fire (IF) or Leaky Integrate-and-Fire (LIF)
- During inference, the SNN processes inputs over multiple discrete time steps instead of a single forward pass.
- Information is encoded in the form of spike rates, i.e., the frequency of neuron firing over time.
- The SNN output gradually approximates the ANN output as the number of time steps increases.
- This approach reduces energy consumption but introduces inference latency due to time-step-based computation.

Thus, **structurally** there is potentially no difference between the ANN and converted SNN (i.e no addition of layers, weight transformation etc) but computations happen only when neurons are spiked over a period of time leading to sparsity in time and event driven nature

$$U_i^{temp}(t) = \lambda U_i(t-1) + \sum_j W_{ij} S_j(t)$$
$$Y_i = \text{clip} \left(\sum_j W_{ij} X_j, 0, \mu \right)$$
$$S_i(t) = \begin{cases} V^{th}, & \text{if } U_i^{temp}(t) > V^{th} \\ 0, & \text{otherwise} \end{cases}$$
$$U_i(t) = U_i^{temp}(t) - S_i(t)$$
$$\bar{S}_t = \frac{V^{th}}{T} \text{clip} \left(\left\lfloor \frac{T}{V^{th}} \mathbf{W}_t \bar{\mathbf{S}}_{t-1} \right\rfloor, 0, T \right),$$

- Standard ReLU has no upper bound, outputs can grow arbitrarily large, making it poorly suited for SNN conversion since the SNN's firing range is inherently bounded by V_{th} (Threshold voltage)
- Threshold ReLU clips the output between 0 and μ , directly mirroring the SNN's bounded firing range, making the two architectures numerically compatible after weight copying

About NengoDL

NengoDL a software framework that bridges the gap between neuromorphic modelling (simulating biological brain networks) and deep learning (conventional neural networks).

```
converter = nengo_dl.Converter(  
    ann_model,  
    swap_activations={tf.keras.activations.relu: nengo.LIF}  
)  
#Converts the trained ANN model into an equivalent SNN and transfers its learned weights.  
  
snn_network = converter.net #Retrieves the converted spiking neural network from the converter.  
  
output_probe = list(converter.outputs.values())[-1] #Gets the probe that records the activity of the final output layer of the SNN.  
  
print("\nANN successfully converted to SNN")
```

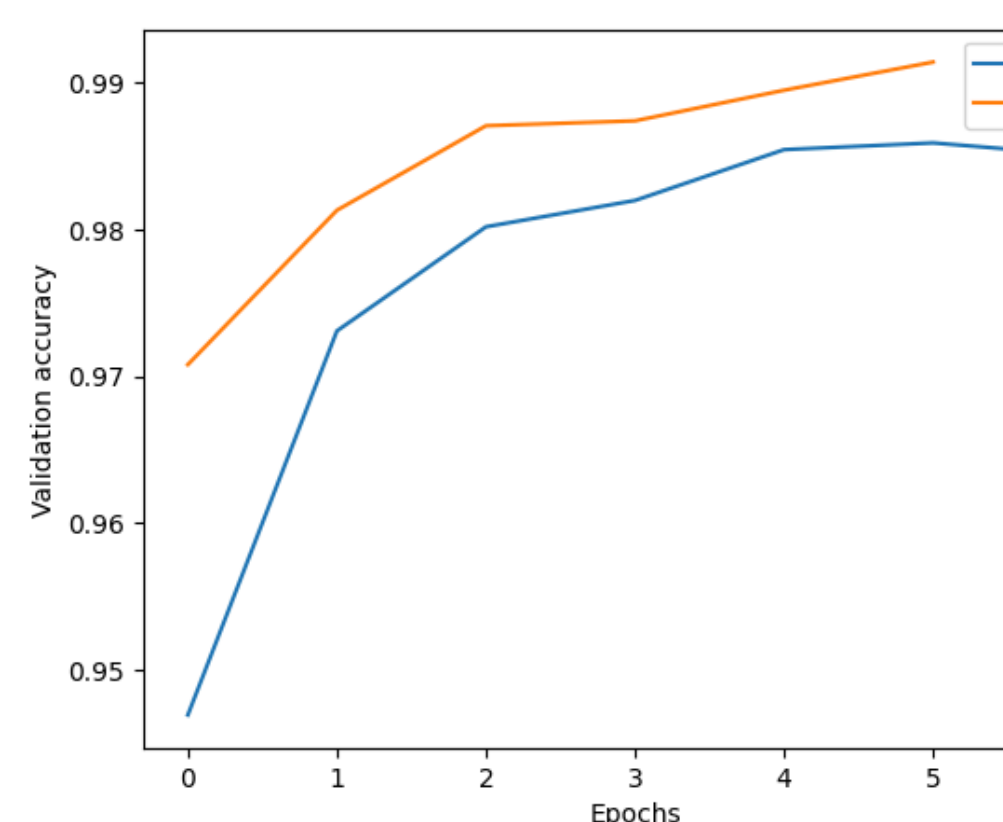
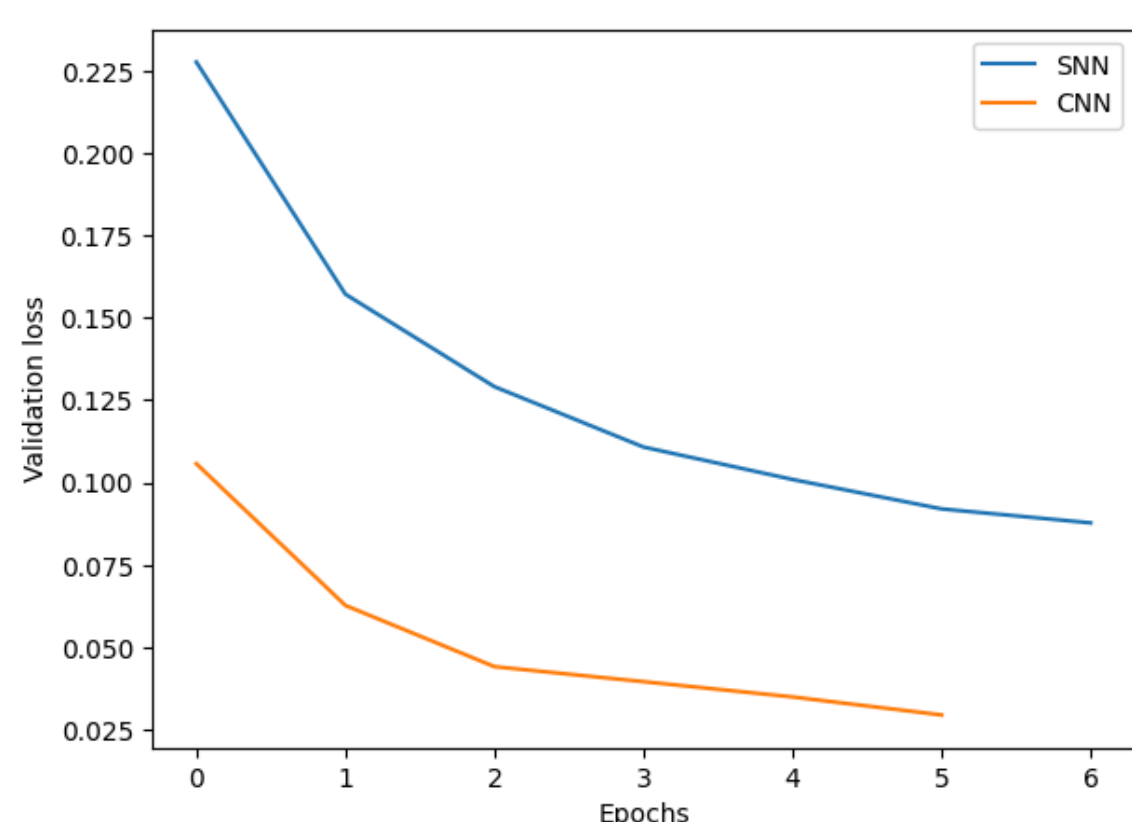
SNN Layers

Layer	Parameters
Linear	input = 9409, output = 1024
Leaky	beta = 0.95
Linear	input = 1024, output = 10
Leaky	beta = 0.95

ANN Layers

Layer	Parameters
Conv2d	input channels = 1, output channels = 16, kernel size = (3, 3), stride = (1, 1), activation = ReLU
MaxPool2d	kernel size = 2, stride = 2, padding = 0
Conv2d	input channels = 16, output channels = 32, kernel size = (3, 3), stride = (1, 1), activation = ReLU
MaxPool2d	kernel size = 2, stride = 2, padding = 0
Linear	input = 15488, output = 1024, activation = ReLU
Linear	input = 1024, output = 10, activation = log softmax

Validation loss and accuracy for digits classification



Result for model on MNIST Dataset

Accuracy of converted SNN: 98.33%

Accuracy of initially trained CNN: 98.41%

Conclusion :

- Current SOTA in DNN-to-SNN conversion focuses on minimizing activation mismatch through threshold optimization and scaling techniques largely preserving the original DNN structure.
- Future work can explore simultaneous architecture compression during conversion - pruning layers, reducing width, or merging redundant connections - since SNNs encode information across both space and time, potentially requiring fewer neurons per layer.

References

- S. Deng, Y. Li, S. Zhang, and S. Gu, "Can deep neural networks be converted to ultra low-latency spiking neural networks?" in Proceedings of the 25th Design, Automation and Test in Europe Conference (DATE), Grenoble, France, 2022, pp. 717-722.
- H. Liu, K. Shi, J. Zhang, Y. Shan, J. Wu, W. Chen, and M. Zhang, "HardF-SNN: Hardware-friendly quantization for spiking neural networks with efficient integer-arithmetic-only inference," Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW), 2023.
- D. Rasmussen, "NengoDL: Combining deep learning and neuromorphic modelling methods," arXiv preprint arXiv:1805.11144, 2018.
- <https://www.researchgate.net/publication/358598741/figure/fig1/AS:1156539024982027@1652751799585/illustrations-of-SNN-model-a-The-process-of-image-input-into-a-SNN-b-A-spiking-neuron.png>